

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 3 Practical No: 5

Roll no: 35 PRN No: 12414739 Name of Student: KHYATI KENDHE

Subject Name : Deep Learning

Problem Statement : Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

Download the SMS Spam Collection (if not already present) and load it into a dataframe with label and message columns.

Encode labels as binary (ham = 0, spam = 1) and compute the word count of each message.

Plot the class balance and message length distribution (EDA).

Tokenize the messages with a Keras Tokenizer (top 5,000 words, out-of-vocabulary token for unseen words).

Convert each message into a sequence of integer token IDs.

Pad or truncate every sequence to a fixed length of 60, and record each message's true length.

Split the data 70/15/15 into train, validation, and test sets, stratified to keep the ~13% spam ratio in every split.

Define one model builder with an identical skeleton: Embedding (with masking of padded positions) → recurrent layer → Dropout → Dense sigmoid output.

Build three models that differ only in the recurrent layer: SimpleRNN, LSTM, and GRU.

Compile all three with the same optimizer (Adam), loss (binary cross-entropy), embedding size, hidden size, and dropout.

Train each model with the same batch size and epoch limit, using early stopping on validation loss, and record the training time.

Plot the training and validation loss and accuracy curves for all three models together.

Predict spam probabilities on the test set and threshold at 0.5 to get class labels, recording inference time.

Compute accuracy, precision, recall, F1-score, and ROC-AUC for each model.

Plot the metrics comparison bar chart.

Plot a confusion matrix for each model.

Plot the overlaid ROC curves.

Bucket the test messages by length and plot each model's accuracy per bucket (robustness check).

Compare parameter count, training time, and inference time across the three models.

Build a normalized summary heatmap and print the best model per criterion, noting that precision, recall, and F1 matter more than accuracy because of the class imbalance.

Code:

```
"""
```

Sequence Classification with RNN, LSTM, and GRU: A Comparative Study

Dataset : SMS Spam Collection (5,572 real, labeled English SMS messages,
ham vs spam) - Almeida & Gomez Hidalgo, UCI ML Repository.

```
"""
```

```
import os
import time
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import (Embedding, SimpleRNN, LSTM, GRU,
                                     Dense, Dropout, Input)
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, roc_curve, roc_auc_score)

# -----
# 1. SETUP & REPRODUCIBILITY
# -----
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```

OUT_DIR = "outputs_rnn_lstm_gru"
os.makedirs(OUT_DIR, exist_ok=True)

sns.set_theme(style="whitegrid", palette="deep", font_scale=1.0)
PALETTE = {"RNN": "#e0554e", "LSTM": "#3d6dbf", "GRU": "#2ea86e"}
plt.rcParams["figure.dpi"] = 120
plt.rcParams["axes.titleweight"] = "bold"

print(f"TensorFlow version: {tf.__version__}")
print(f"Random seed: {SEED}")

# -----
# 2. LOAD & EXPLORE THE DATASET
# -----
DATA_PATH = "sms_spam.tsv"
DATA_URL = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"

if not os.path.exists(DATA_PATH):
    import urllib.request
    print(f"'{DATA_PATH}' not found locally - downloading from {DATA_URL} ...")
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

df = pd.read_csv(DATA_PATH, sep="\t", header=None, names=["label", "message"])
df["target"] = (df["label"] == "spam").astype(int)
df["word_count"] = df["message"].str.split().str.len()

print(f"\nTotal messages: {len(df)}")
print(df["label"].value_counts())
print(df["word_count"].describe())

fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
class_counts = df["label"].value_counts()
axes[0].bar(class_counts.index, class_counts.values,
            color=["#2ea86e", "#e0554e"], edgecolor="white")
axes[0].set_title("Class Balance (Ham vs Spam)")
axes[0].set_ylabel("Count")
for i, v in enumerate(class_counts.values):
    axes[0].text(i, v + 40, str(v), ha="center", fontweight="bold")

axes[1].hist(df["word_count"], bins=40, color="#5b7fd6", edgecolor="white")
axes[1].set_title("Message Length Distribution")
axes[1].set_xlabel("Words per message")
axes[1].set_ylabel("Count")
plt.tight_layout()
fig.savefig(f"{OUT_DIR}/01_eda_overview.png")
plt.close(fig)

# -----
# 3. PREPROCESSING (tokenize, pad, split)

```

```

# -----
VOCAB_SIZE = 5000
MAX_LEN = 60

tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>")
tokenizer.fit_on_texts(df["message"])

sequences = tokenizer.texts_to_sequences(df["message"])
true_lengths = np.array([min(len(s), MAX_LEN) for s in sequences])
X = pad_sequences(sequences, maxlen=MAX_LEN, padding="post", truncating="post")
y = df["target"].values

# 70 / 15 / 15 stratified split (stratify preserves the ~13% spam ratio
# in every split, which matters given the class imbalance)
X_train, X_temp, y_train, y_temp, len_train, len_temp = train_test_split(
    X, y, true_lengths, test_size=0.30, random_state=SEED, stratify=y)
X_val, X_test, y_val, y_test, len_val, len_test = train_test_split(
    X_temp, y_temp, len_temp, test_size=0.50, random_state=SEED, stratify=y_temp)

print(f"\nVocabulary size (capped): {VOCAB_SIZE}")
print(f"Train / Val / Test sizes: {len(X_train)} / {len(X_val)} / {len(X_test)}")
print(f"Spam ratio -> train: {y_train.mean():.3f} val: {y_val.mean():.3f} test: {y_test.mean():.3f}")

# -----
# 4. MODEL ARCHITECTURES (RNN / LSTM / GRU)
# -----
# All three share an IDENTICAL skeleton:
# token ids -> Embedding(mask_zero=True) -> [RNN|LSTM|GRU] -> Dropout -> Dense(sigmoid)
# Embedding's mask propagates through the recurrent layer, so padded
# positions (id=0) are skipped rather than polluting the final hidden
# state - the Keras equivalent of PyTorch's pack_padded_sequence.
EMBED_DIM = 32
HIDDEN_DIM = 64
DROPOUT = 0.3

def build_model(rnn_type):
    assert rnn_type in ("RNN", "LSTM", "GRU")
    recurrent_layer = {
        "RNN": SimpleRNN(HIDDEN_DIM),
        "LSTM": LSTM(HIDDEN_DIM),
        "GRU": GRU(HIDDEN_DIM),
    }[rnn_type]

    model = keras.Sequential([
        Input(shape=(MAX_LEN,)),
        Embedding(input_dim=VOCAB_SIZE, output_dim=EMBED_DIM, mask_zero=True),
        recurrent_layer,
        Dropout(DROPOUT),
        Dense(1, activation="sigmoid"),
    ], name=f"{rnn_type}_classifier")

```

```

model.compile(optimizer=keras.optimizers.Adam(learning_rate=2e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
return model

models = {name: build_model(name) for name in ["RNN", "LSTM", "GRU"]}

param_table = pd.DataFrame({
    "Model": list(models.keys()),
    "Total Parameters": [m.count_params() for m in models.values()],
})
print("\n", param_table.to_string(index=False))

# -----
# 5. TRAINING LOOP
# -----
EPOCHS = 25
BATCH_SIZE = 32

histories = {}
training_time = {}

for name, model in models.items():
    print(f"\n{' '*70}\nTraining {name}\n{' '*70}")
    early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)
    t0 = time.time()
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        callbacks=[early_stop], verbose=2,
    )
    training_time[name] = time.time() - t0
    histories[name] = history.history
    print(f"Total training time for {name}: {training_time[name]:.2f}s")

# -----
# 6. TRAINING CURVES
# -----
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
for name, h in histories.items():
    c = PALETTE[name]
    axes[0, 0].plot(h["loss"], label=name, color=c, marker="o", ms=3)
    axes[0, 1].plot(h["val_loss"], label=name, color=c, marker="o", ms=3)
    axes[1, 0].plot(h["accuracy"], label=name, color=c, marker="o", ms=3)
    axes[1, 1].plot(h["val_accuracy"], label=name, color=c, marker="o", ms=3)

titles = ["Training Loss", "Validation Loss", "Training Accuracy", "Validation Accuracy"]
ylabels = ["BCE Loss", "BCE Loss", "Accuracy", "Accuracy"]
for ax, title, ylab in zip(axes.flat, titles, ylabels):

```

```

    ax.set_title(title); ax.set_xlabel("Epoch"); ax.set_ylabel(ylabel); ax.legend()
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/02_training_curves.png')
plt.close(fig)

# -----
# 7. TEST-SET EVALUATION: FULL METRIC SUITE
# -----
test_results = {}
for name, model in models.items():
    t0 = time.time()
    y_prob = model.predict(X_test, verbose=0).flatten()
    inf_time = time.time() - t0
    y_pred = (y_prob >= 0.5).astype(int)
    test_results[name] = dict(y_true=y_test, y_pred=y_pred, y_prob=y_prob,
                             lengths=len_test, inference_time=inf_time)

metrics_rows = []
for name, r in test_results.items():
    y_true, y_pred, y_prob = r["y_true"], r["y_pred"], r["y_prob"]
    metrics_rows.append({
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1-score": f1_score(y_true, y_pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_true, y_prob),
        "Params": models[name].count_params(),
        "Train Time (s)": training_time[name],
        "Inference Time (s)": r["inference_time"],
    })

results_df = pd.DataFrame(metrics_rows).set_index("Model").loc[["RNN", "LSTM", "GRU"]]
print("\n--- Test-Set Performance Summary ---")
print(results_df.round(4).to_string())

metrics_to_plot = ["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"]
plot_df = results_df[metrics_to_plot].reset_index().melt(
    id_vars="Model", var_name="Metric", value_name="Score")

fig = plt.figure(figsize=(11, 5))
sns.barplot(data=plot_df, x="Metric", y="Score", hue="Model", palette=PALETTE)
plt.ylim(0, 1.05)
plt.title("Test-Set Metrics: RNN vs LSTM vs GRU (Spam = Positive Class)")
for container in plt.gca().containers:
    plt.gca().bar_label(container, fmt="%.3f", fontsize=8, padding=2)
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/03_metrics_comparison.png')
plt.close(fig)

# -----

```

8. CONFUSION MATRICES & ROC CURVES

```
# -----
fig, axes = plt.subplots(1, 3, figsize=(14, 4.2))
for ax, name in zip(axes, ["RNN", "LSTM", "GRU"]):
    cm = confusion_matrix(test_results[name]["y_true"], test_results[name]["y_pred"])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["Ham (0)", "Spam (1)"], yticklabels=["Ham (0)", "Spam (1)"])
    ax.set_title(f'{name} - Confusion Matrix')
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/04_confusion_matrices.png')
plt.close(fig)

fig = plt.figure(figsize=(6.5, 6))
for name, r in test_results.items():
    fpr, tpr, _ = roc_curve(r["y_true"], r["y_prob"])
    auc = roc_auc_score(r["y_true"], r["y_prob"])
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})', color=PALETTE[name], linewidth=2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random Chance")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
plt.title("ROC Curves - RNN vs LSTM vs GRU")
plt.legend(loc="lower right")
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/05_roc_curves.png')
plt.close(fig)
```

9. ROBUSTNESS CHECK: ACCURACY VS. MESSAGE LENGTH

```
# -----
# Real messages vary naturally in length (1-171 words). This checks
# whether any model's accuracy degrades on longer messages, which would
# hint at difficulty carrying information across more time-steps -
# the practical, real-data analogue of a long-range-dependency test.
bins = [0, 8, 15, 25, MAX_LEN]
bin_labels = ["1-8", "9-15", "16-25", f"26-{MAX_LEN}+"]
```

```
bucket_rows = []
for name, r in test_results.items():
    bucket_idx = np.digitize(r["lengths"], bins[1:-1], right=True)
    for b, blabel in enumerate(bin_labels):
        mask = bucket_idx == b
        if mask.sum() == 0:
            continue
        acc = accuracy_score(r["y_true"][mask], r["y_pred"][mask])
        bucket_rows.append({"Model": name, "Length Bucket": blabel,
                           "Accuracy": acc, "N": int(mask.sum())})
```

```
bucket_df = pd.DataFrame(bucket_rows)
```

```
fig = plt.figure(figsize=(8.5, 5.5))
for name in ["RNN", "LSTM", "GRU"]:
```

```

    sub = bucket_df[bucket_df["Model"] == name]
    plt.plot(sub["Length Bucket"], sub["Accuracy"], marker="o", linewidth=2.5,
             markersize=8, label=name, color=PALETTE[name])
plt.ylim(0.5, 1.02)
plt.xlabel("Message Length Bucket (words)"); plt.ylabel("Test Accuracy")
plt.title("Robustness Check: Accuracy vs. Message Length")
plt.legend(title="Model")
plt.axhline(y_test.mean() if y_test.mean() > 0.5 else 1 - y_test.mean(),
            color="gray", linestyle=":", label="Majority-class baseline")
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/06_length_stress_test.png')
plt.close(fig)

# -----
# 10. COMPUTATIONAL EFFICIENCY
# -----

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
names = ["RNN", "LSTM", "GRU"]
colors = [PALETTE[n] for n in names]

axes[0].bar(names, [models[n].count_params() for n in names], color=colors, edgecolor="white")
axes[0].set_title("Total Trainable Parameters")
axes[0].yaxis.set_major_formatter(mticker.StrMethodFormatter("{x:,.0f}"))

axes[1].bar(names, [training_time[n] for n in names], color=colors, edgecolor="white")
axes[1].set_title("Total Training Time")
axes[1].set_ylabel("Seconds")

axes[2].bar(names, [test_results[n]["inference_time"] for n in names], color=colors, edgecolor="white")
axes[2].set_title(f'Test-Set Inference Time (n={len(X_test)})')
axes[2].set_ylabel("Seconds")

for ax in axes:
    for p in ax.patches:
        val = p.get_height()
        label = f'{val:,.2f}' if val < 1000 else f'{val:,.0f}'
        ax.annotate(label, (p.get_x() + p.get_width() / 2, val),
                    ha="center", va="bottom", fontsize=9)
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/07_efficiency.png')
plt.close(fig)

print(f'\nAll figures saved to ./{OUT_DIR}/')
print("Done.")

```

OUTPUT:

TensorFlow version: 2.20.0
Random seed: 42
'sms_spam.tsv' not found locally - downloading from <https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv> ...

Total messages: 5572
label
ham 4825
spam 747
Name: count, dtype: int64
count 5572.000000
mean 15.597452
std 11.404053
min 1.000000
25% 7.000000
50% 12.000000
75% 23.000000
max 171.000000
Name: word_count, dtype: float64

Vocabulary size (capped): 5000
Train / Val / Test sizes: 3900 / 836 / 836
Spam ratio -> train: 0.134 val: 0.134 test: 0.134

Model	Total Parameters
RNN	166273
LSTM	184897
GRU	178881

Training RNN

```
=====
Epoch 1/25
122/122 - 13s - 103ms/step - accuracy: 0.9013 - loss: 0.2626 - val_accuracy: 0.9785 - val_loss: 0.0726
Epoch 2/25
122/122 - 3s - 23ms/step - accuracy: 0.9869 - loss: 0.0492 - val_accuracy: 0.9797 - val_loss: 0.0753
Epoch 3/25
122/122 - 3s - 24ms/step - accuracy: 0.9951 - loss: 0.0195 - val_accuracy: 0.9844 - val_loss: 0.0623
Epoch 4/25
122/122 - 3s - 23ms/step - accuracy: 0.9990 - loss: 0.0059 - val_accuracy: 0.9904 - val_loss: 0.0566
Epoch 5/25
122/122 - 4s - 34ms/step - accuracy: 0.9997 - loss: 0.0023 - val_accuracy: 0.9844 - val_loss: 0.0654
Epoch 6/25
122/122 - 3s - 22ms/step - accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 0.9880 - val_loss: 0.0620
Epoch 7/25
122/122 - 3s - 23ms/step - accuracy: 1.0000 - loss: 9.3145e-04 - val_accuracy: 0.9856 - val_loss: 0.0671
Epoch 8/25
122/122 - 6s - 48ms/step - accuracy: 1.0000 - loss: 6.9932e-04 - val_accuracy: 0.9892 - val_loss: 0.0655
Epoch 9/25
122/122 - 3s - 28ms/step - accuracy: 1.0000 - loss: 5.5495e-04 - val_accuracy: 0.9880 - val_loss: 0.0654
Total training time for RNN: 40.10s
```

Training LSTM

```
=====
Epoch 1/25
122/122 - 7s - 61ms/step - accuracy: 0.9369 - loss: 0.2043 - val_accuracy: 0.9821 - val_loss: 0.0563
Epoch 2/25
122/122 - 6s - 52ms/step - accuracy: 0.9892 - loss: 0.0405 - val_accuracy: 0.9785 - val_loss: 0.0683
Epoch 3/25
122/122 - 5s - 40ms/step - accuracy: 0.9959 - loss: 0.0177 - val_accuracy: 0.9892 - val_loss: 0.0541
Epoch 4/25
122/122 - 6s - 52ms/step - accuracy: 0.9990 - loss: 0.0062 - val_accuracy: 0.9892 - val_loss: 0.0465
Epoch 5/25
122/122 - 9s - 75ms/step - accuracy: 0.9992 - loss: 0.0031 - val_accuracy: 0.9868 - val_loss: 0.0628
Epoch 6/25
122/122 - 6s - 52ms/step - accuracy: 0.9987 - loss: 0.0054 - val_accuracy: 0.9868 - val_loss: 0.0468
Epoch 7/25
122/122 - 5s - 40ms/step - accuracy: 0.9995 - loss: 0.0024 - val_accuracy: 0.9880 - val_loss: 0.0528
Epoch 8/25
122/122 - 6s - 52ms/step - accuracy: 0.9995 - loss: 0.0024 - val_accuracy: 0.9868 - val_loss: 0.0584
Epoch 9/25
```

Training GRU
Epoch 1/25
122/122 - 10s - 81ms/step - accuracy: 0.9262 - loss: 0.2162 - val_accuracy: 0.9856 - val_loss: 0.0591
Epoch 2/25
122/122 - 5s - 43ms/step - accuracy: 0.9882 - loss: 0.0422 - val_accuracy: 0.9904 - val_loss: 0.0479
Epoch 3/25
122/122 - 10s - 84ms/step - accuracy: 0.9962 - loss: 0.0115 - val_accuracy: 0.9904 - val_loss: 0.0512
Epoch 4/25
122/122 - 6s - 49ms/step - accuracy: 0.9987 - loss: 0.0042 - val_accuracy: 0.9833 - val_loss: 0.0808
Epoch 5/25
122/122 - 6s - 49ms/step - accuracy: 0.9990 - loss: 0.0045 - val_accuracy: 0.9892 - val_loss: 0.0624
Epoch 6/25
122/122 - 5s - 43ms/step - accuracy: 0.9995 - loss: 0.0021 - val_accuracy: 0.9856 - val_loss: 0.0687
Epoch 7/25
122/122 - 10s - 84ms/step - accuracy: 1.0000 - loss: 7.3247e-04 - val_accuracy: 0.9868 - val_loss: 0.0792
Total training time for GRU: 52.94s

--- Test-Set Performance Summary ---

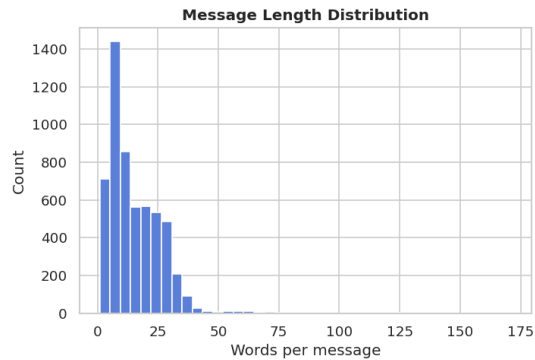
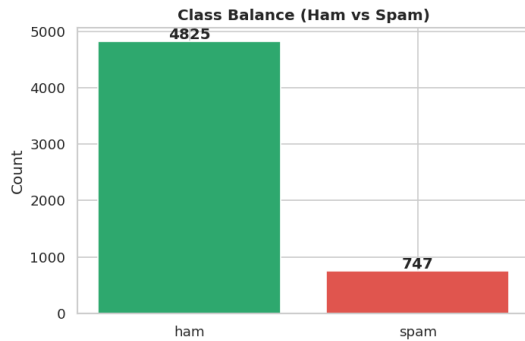
	Accuracy	Precision	Recall	F1-score	ROC-AUC	Params	Train Time (s)	Inference Time (s)
Model								
RNN	0.9892	0.9813	0.9375	0.9589	0.9839	166273	40.0991	0.5049
LSTM	0.9880	0.9722	0.9375	0.9545	0.9906	184897	56.5692	0.6930
GRU	0.9868	1.0000	0.9018	0.9484	0.9891	178881	52.9415	1.0935

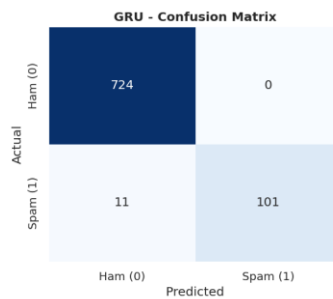
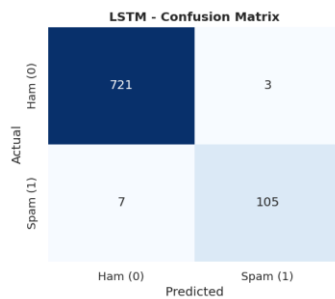
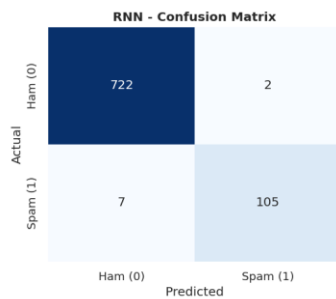
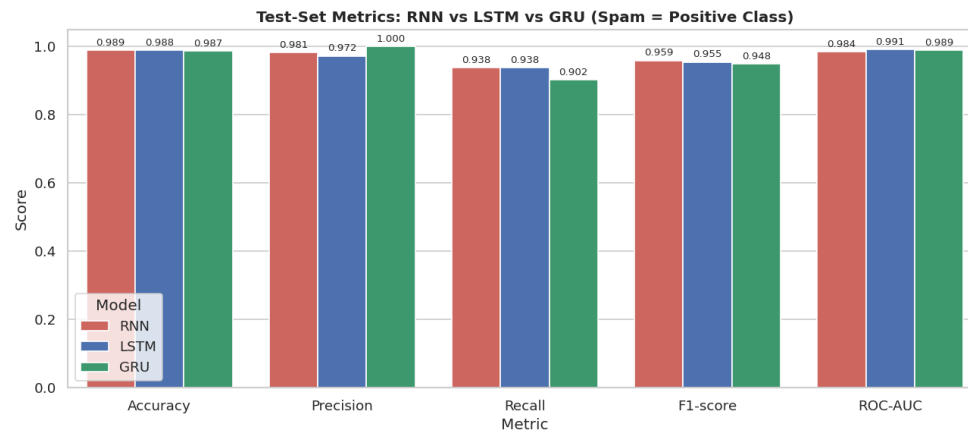
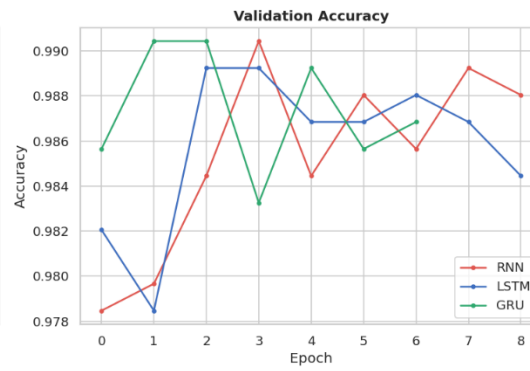
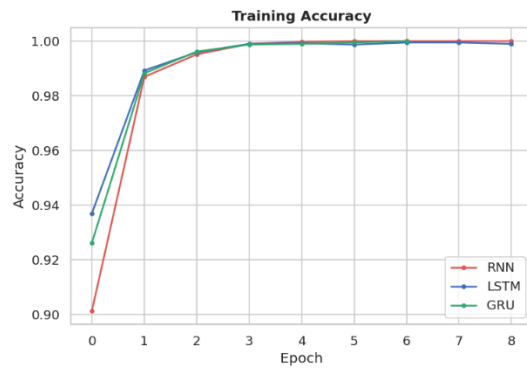
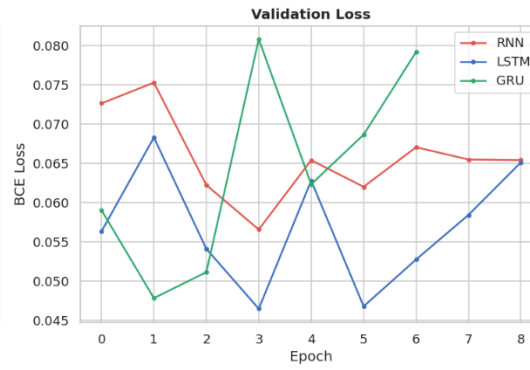
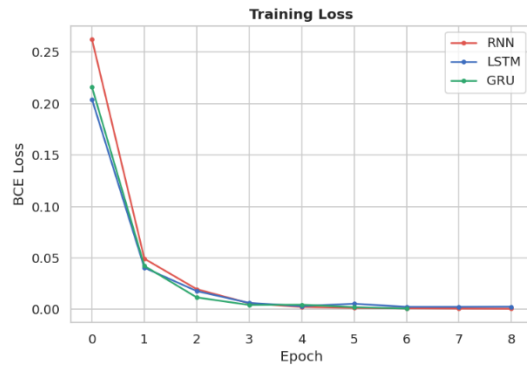
SUMMARY OF THIS RUN

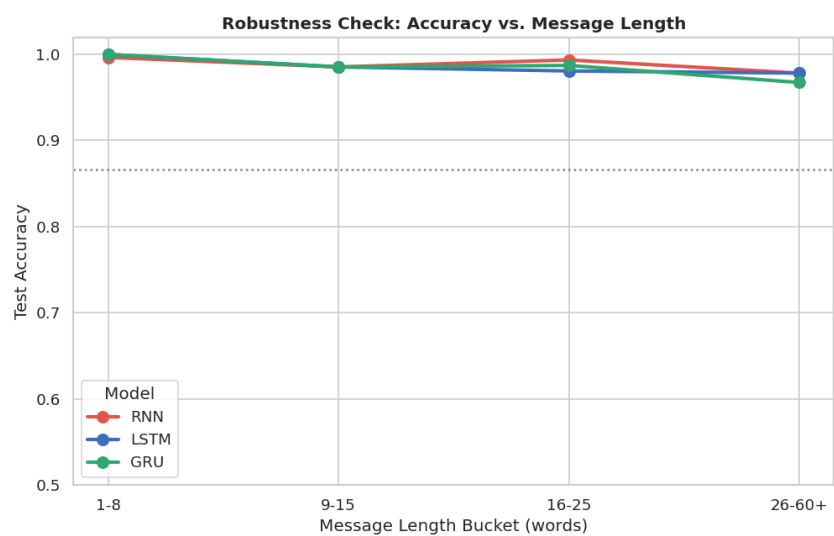
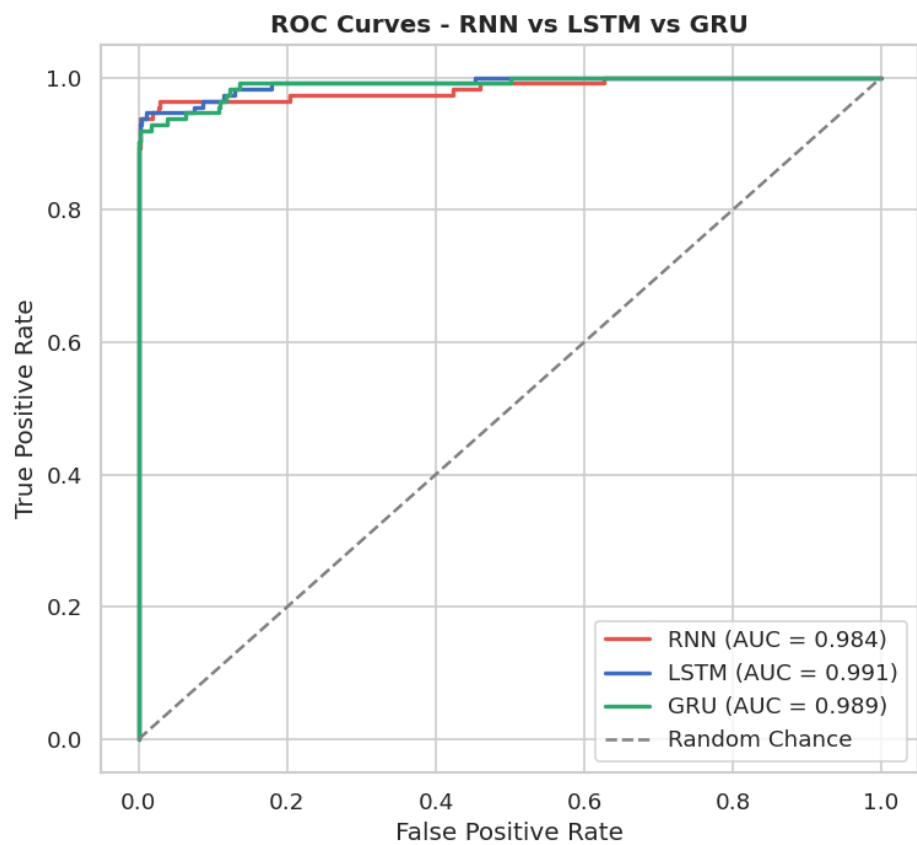
RNN	Acc: 0.989	Precision: 0.981	Recall: 0.938	F1: 0.959	AUC: 0.984	Params: 166,273
LSTM	Acc: 0.988	Precision: 0.972	Recall: 0.938	F1: 0.955	AUC: 0.991	Params: 184,897
GRU	Acc: 0.987	Precision: 1.000	Recall: 0.902	F1: 0.948	AUC: 0.989	Params: 178,881

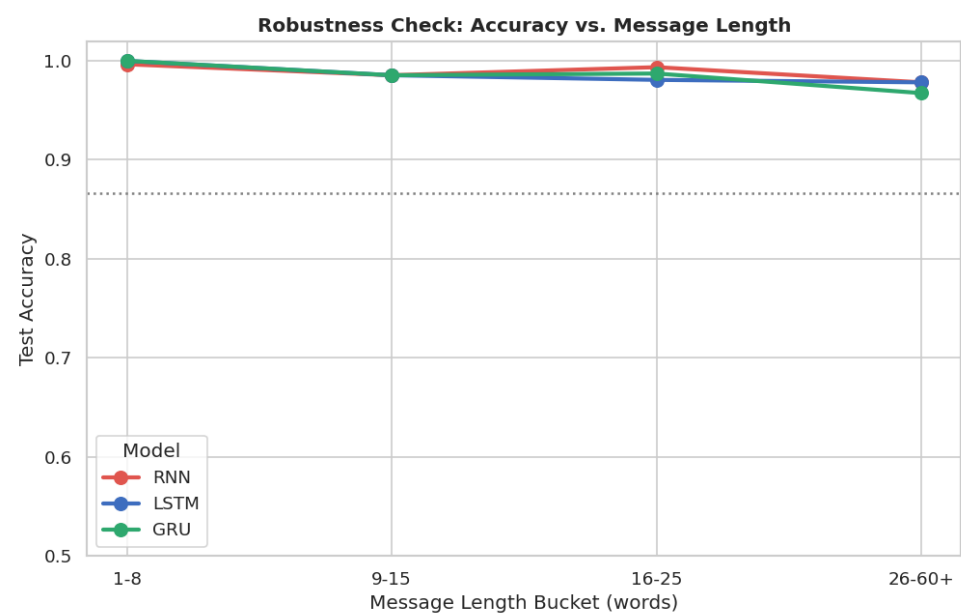
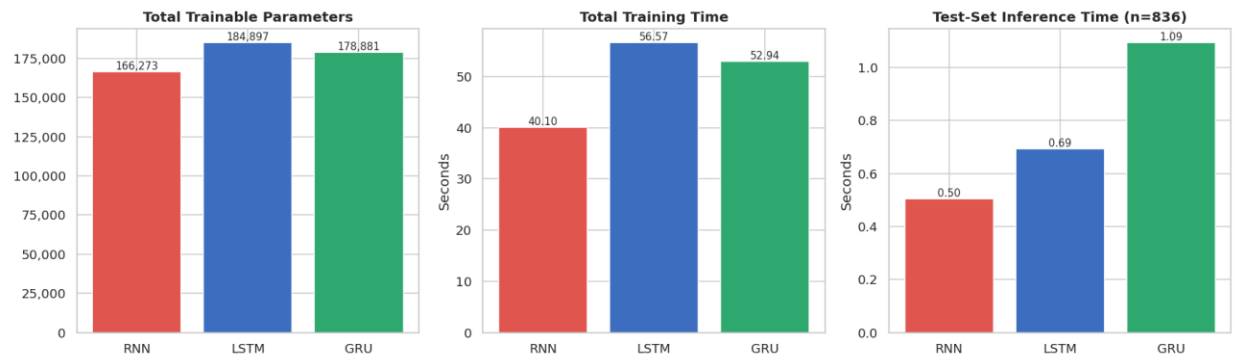
Best F1-score : RNN (0.9589)
Best ROC-AUC : LSTM (0.9906)
Fewest parameters: RNN (166,273)
Fastest training : RNN (40.10s)

Note: with ~13.4% spam in the data, accuracy alone can be misleading - Precision/Recall/F1 on the SPAM (minority, positive) class are the more trustworthy comparison signal here.









Normalized Performance Heatmap (greener = better)							
Model	RNN	0.989	0.959	0.984	166273.0	40.099	0.505
	LSTM	0.988	0.955	0.991	184897.0	56.569	0.693
	GRU	0.987	0.948	0.989	178881.0	52.942	1.093
		Accuracy	F1-score	ROC-AUC	Params	Train Time (s)	Inference Time (s)

